



Faculty of Engineering & Technology
Electrical and Computer Engineering Department
Advanced Architecture – ENCS5331
Course Project – Phase #1
Background & Literature Review

Prepared By :

Kareem Hamza	1220708
Noor Battat	1210075
Anas Sarabta	1200242

Instructor : Dr. Ayman Hroub

Date : 14/12/2025

2. Background

Retrieval-Augmented Generation (RAG)

- Retrieval-Augmented Generation couples information retrieval with neural text generation: a retriever selects documents relevant to a user query from a large corpus, and a generator conditions on those retrieved passages to produce grounded outputs. The retriever is typically implemented as a nearest-neighbour search over precomputed embeddings; the generator is a transformer-based language model that incorporates retrieved context during decoding. The net effect is improved factuality and extensibility of language models, but achieving this in interactive settings places strict latency and throughput requirements on the retrieval subsystem.

Retrieval Modalities : ENNS & ANNS

- Two broad retrieval strategies appear in practice. Exact nearest-neighbor search (ENNS) computes true similarity across stored vectors and yields the highest recall, but scales poorly as corpus size increases. Approximate nearest-neighbor search (ANNS) reduces query cost using quantization, graph indices, or product quantization; this improves throughput but can degrade recall and often requires costly re-ranking. Separately, richer retrieval models—token-level late interaction (ColBERT) or hybrid sparse–dense schemes (SPLADE)—increase retrieval quality by enabling finer-grained matching, but they also inflate memory footprint and per-query access complexity. These algorithmic choices therefore define different, and often competing, hardware demands.

The Memory Compute Mismatch

- From a hardware perspective the retrieval kernel is low in arithmetic intensity: many simple dot products over large vectors, where the main cost is fetching vector elements from memory. Modern CPUs and GPUs are optimized for compute-heavy kernels; when the limiting factor is DRAM bandwidth and latency, arithmetic units sit idle while memory requests stream. Moreover, deep cache hierarchies provide little benefit when vectors are streaming with low reuse: cache fills and coherence overheads can even reduce efficiency. Thus the dominant system cost for RAG retrieval is data movement, not raw compute.

Architectural Responses : PIM, Near-Memory, Near-Data & Disaggregated Memory

- To address the data-movement problem, researchers pursue three complementary architectural responses. Processing-in-memory (PIM) embeds compute inside or beside DRAM banks, minimizing off-chip transfers for kernels that map to simple primitives. Near-memory accelerators colocate small processing tiles with memory packages and expose them via standards such as CXL (enabling CXL.mem and CXL.cache semantics). Near-data or in-storage compute pushes lightweight logic into SSD controllers to accelerate workloads that cannot fit in DRAM. Each approach trades programmability, precision, thermal and area constraints, and deployment feasibility; selecting among them requires matching retrieval algorithms to architectural primitives, not just raw throughput numbers.

Design Gap

- The combined algorithmic evolution (denser interaction models, hybrid retrieval) and the emergence of memory-centric hardware create an opportunity — and a difficulty. The opportunity is that placing the right primitives near data can collapse latency and energy for retrieval; the difficulty is that existing hardware proposals address fragments of the problem (ENNS vs ANNS, DRAM vs storage, or specialized primitives) without a unified co-design that supports rich retrieval models across a multi-tier memory hierarchy. This project targets that gap: align retrieval algorithm patterns with memory-centric accelerators so high-quality RAG becomes practical at scale.

LITERATURE REVIEW

(Core Papers)

1. Intelligent Knowledge Store (IKS) — [Accelerating Retrieval-Augmented Generation](#)

Summary: IKS proposes a type-2 CXL memory-expander device that embeds near-memory accelerators to run *exact* nearest-neighbor search (ENNS) inside a large CXL-attached memory module. The authors argue that for RAG pipelines exact search can reduce downstream generator load and end-to-end latency, and they demonstrate large speedups (e.g., tens of $\times$ versus CPU for exhaustive search on a 512 GB vector DB) in their evaluations. IKS emphasizes a cache-coherent interface so the host can use module DRAM without wasting server DRAM, and it shows concrete RAG end-to-end improvements while discussing scaling and aggregation bottlenecks when many modules participate.

Related work: IKS sits between two directions in the literature: in-memory / near-memory accelerators that move compute closer to large datasets (e.g., PIM and NDP designs), and system-level RAG work that treats retrieval as a first-class, latency-sensitive stage. Compared with PIM proposals that target approximate ANNS (e.g., MemANNS), IKS focuses on *exact* search in coherent memory modules and evaluates the full RAG pipeline, which makes it especially relevant for hardware designs that must supply high-quality context to LLMs.

2. MemANNS — [Mapping ANNS to Practical PIM \(UPMEM DPUs\)](#)

Summary: MemANNS studies how to map large-scale ANNS kernels (for example, index + IVF/PQ-like flows) to a practical, programmable PIM platform (UPMEM DPUs). It proposes data-placement, quantization, and query-scheduling strategies that balance memory footprint, per-DPU compute, and cross-chip bandwidth to achieve good recall/latency tradeoffs at billion-vector scales. The paper presents end-to-end experiments on real PIM hardware (or realistic PIM simulators) showing energy and latency improvements over CPU/GPU baselines for several ANNS configurations.

Related work: MemANNS is firmly in the practical PIM camp: it borrows PIM programming/placement lessons (e.g., from UPMEM analyses) and adapts commonly used ANNS tricks (quantization, sharding) to the constraints of DPUs. Unlike fully analog or SSD-based approaches, MemANNS emphasizes portability to existing programmable PIM hardware and thus provides a realistic bridge from lab PIM proposals to usable RAG backends. This work complements IKS (which targets exact search in CXL modules) by showing how *approximate* ANNS pipelines can be engineered on real PIM devices.

3. NDSEARCH — [Near-Data Processing for Graph-Traversal ANNS](#)

Summary: NDSEARCH targets graph-based ANNS (e.g., HNSW-style traversal), an algorithm family with high recall but heavy pointer chasing and large memory requirements. The paper presents a hardware-software co-design that offloads parts of the traversal to an in-storage compute architecture (SEARSSD) and a tailored processing model that exploits NAND LUN-level parallelism and two-level scheduling to maximize internal SSD bandwidth. NDSEARCH reports very large throughput and energy gains compared to CPU/GPU baselines and other smart-storage designs, while carefully describing limitations coming from controller compute and flash latency.

Related work: NDSEARCH aligns with the NDP/SmartSSD line (moving irregular, pointer-heavy workloads closer to storage) and contrasts with PIM approaches that assume ample compute near DRAM. It is especially relevant when indices exceed DRAM capacity: where MemANNS or IKS assume memory-resident or CXL-attached data, NDSEARCH shows how graph traversal can still be practical on SSDs by reshaping the algorithm and scheduler to the medium. For hardware RAG designs, NDSEARCH suggests viable paths when cost prevents scaling main memory.

(Supporting Papers)

4. Ambit — [In-DRAM bulk bitwise operations](#)

Summary: Ambit demonstrates that commodity DRAM can be orchestrated to perform very fast bulk bitwise operations (AND/OR/NOT) inside DRAM arrays by exploiting analog behavior and simple control sequences; this yields large speed and energy gains for bitset workloads.

Related work: Ambit is an influential proof that complex micro-operations can live in DRAM; while its operations are bitwise (not floating-point dot products), its design and evaluation inform how to expose small but high-value primitives inside memory modules — useful background when considering what minimal in-memory kernels a RAG accelerator should support.

5. RowClone — [In-DRAM bulk copy & initialization](#)

Summary: RowClone proposes low-cost DRAM mechanisms to copy or zero large memory regions completely inside DRAM without moving data over the memory channel; it achieves big energy and latency savings for bulk data-movement tasks.

Related work: RowClone complements Ambit by showing that even simple in-DRAM data-movement primitives can yield measurable system gains. For a RAG hardware design, RowClone motivates supporting fast memory initialization and local copies inside modules to reduce host traffic and speed up index rebuilds or buffering.

6. UPMEM (programmable PIM / DPUs) — [real, deployable PIM](#)

Summary: UPMEM is a commercial PIM platform that integrates small in-order cores (DPUs) with DRAM, enabling programmers to port data-intensive kernels to run on the memory chip and exploit high internal bandwidth. Characterization studies and community benchmarks show that UPMEM delivers important memory-bound gains for suitable workloads while requiring careful code/placement to hide DPU limitations.

Related work: UPMEM is the closest practical PIM baseline to MemANNS. Where many PIM papers are simulation-based, UPMEM offers an available testbed and shows the real software engineering tradeoffs required to benefit from PIM in production RAG stacks.

7. PUMA / ISAAC — [Analog in-memory crossbars for dot-products](#)

Summary: PUMA and ISAAC explore memristor/analog crossbar arrays that perform matrix-vector multiplication extremely efficiently in area and energy, useful for dense dot-product kernels common in neural and vector similarity workloads. The works show major theoretical gains but note integration, precision, and variability challenges.

Related work: These analog approaches represent an orthogonal extreme to PIM/DRAM NDP: they deliver very high raw MAC density but complicate system design (accuracy, analog-to-digital conversion, endurance). For RAG hardware choices, PUMA/ISAAC help bound the efficiency frontier but are less practical for off-the-shelf deployment than programmable PIM or SSD offload today.

8. DiskANN — [SSD-aware billion-scale ANN indexing](#)

Summary: DiskANN presents a graph-based index and software stack that enables billion-point ANNS on a single node with limited RAM by carefully organizing index layout and I/O so that SSDs (not DRAM) store most data while maintaining high recall and interactive latency.

Related work: DiskANN is a software-first way to scale index size cheaply; it complements NDSEARCH by showing how algorithm and I/O engineering alone can get close to acceptable performance before moving to hardware offload. It is a useful baseline when comparing whether to buy specialized NDP/PIM hardware or further optimize the host+SSD stack.

9. FAISS — [Practical ANN library and engineering](#)

Summary: FAISS (Meta) is a production ANN library offering IVF, PQ, HNSW, and GPU primitives; it codifies many practical engineering choices for billion-scale search and remains the dominant software baseline for evaluation and comparison.

Related work: FAISS is the de-facto software standard used to compare PIM, NDP, and other hardware approaches. When presenting hardware gains (e.g., IKS, MemANNS), authors usually measure against FAISS baselines to show real-world speedups under comparable indexing and compression choices.

10. HNSW — [Hierarchical Navigable Small World graphs](#)

Summary: HNSW is a graph-based ANNS algorithm that builds a multi-level small-world graph to reach high recall with low latency; its traversal is efficient in CPU implementations but is pointer-heavy and irregular, making it challenging to accelerate on SIMD and DRAM-friendly substrates.

Related work: HNSW is often paired with DiskANN/FAISS in performance studies (DiskANN for storage scaling, FAISS for baseline implementations) and is precisely the traversal style targeted by NDSEARCH's SSD/NDP approach. Understanding HNSW's irregularity is essential when arguing for near-data or in-controller acceleration in a RAG hardware design.

Comparison Table

Paper	Type	Primary goal	Retrieval type	Hardware modality	Key strengths	Key limitations
IKS — Intelligent Knowledge Store	HW	CXL-attached near-memory ENNS accelerator for RAG	ENNS (exact)	CXL device with LPDDR + NMAs	Large simulated end-to-end speedups; cache-coherent host integration	No silicon prototype; high internal bandwidth; host aggregation scaling issues
MemANNS	HW	Map ANN kernels into PIM to compute distances inside memory	ANNS (approx)	Bank-level PIM (DRAM + compute)	Reduces off-chip transfers; energy/latency gains for ANN	Requires specialized memory modules; approximate recall tradeoffs
NDSEARCH	HW	Offload graph-traversal (HNSW) to SSD controllers for billion-scale search	ANNS (graph)	SSD controller / near-storage compute	Scales beyond DRAM; reduces PCIe/host traffic	Flash latency, limited device compute, index maintenance complexity
Ambit	HW	In-DRAM bitwise operations using commodity DRAM behavior	Not retrieval-specific (primitive)	DRAM-row operation sequencing	Demonstrates in-DRAM compute feasibility and bandwidth savings	Limited to bitwise primitives; not directly dot-product friendly
RowClone	HW	In-DRAM bulk copy	Not retrieval-specific	DRAM internal	Big energy/latency	Not a compute primitive for

		& initialization on primitives	(primitive)	row moves	reduction for bulk moves	similarity; narrow scope
UPMEM (programmable PIM)	HW	Commercial programmable PIM (DPUs inside memory)	Can implement ENNS/ANNs kernels in software	DPUs embedded in memory modules	Practical, programmable PIM platform; real throughput data	DPU perf/power limits; software porting required
PUMA / ISAAC (analog in-memory)	HW	Memristor crossbar accelerators for dense MV/TV ops (DNN)	ENNS-like dot-products (analog)	Analog crossbar / ReRAM	Very high density and energy for dot-products	Precision/variability; integration complexity; not general-purpose
DiskANN	SW/HW (system)	SSD-aware billion-scale ANN index & I/O for sub-second queries	ANNS	SSD-aware software + I/O optimizations	Enables billion-point search on commodity hardware	SSD access far slower than DRAM; dependent on index locality
FAISS	SW	Production ANN library (IVF, PQ, GPU support)	ANNS	GPU/CPU software (no special HW)	Flexible baseline; many algorithmic knobs; GPU accel.	Data movement & GPU memory limits for huge corpora
HNSW	SW (algorithm)	Graph-based ANN index with excellent recall/latency	ANNS (graph)	Irregular-memory pattern on CPU/GPU	High recall, low-latency in practice	Irregular access (pointer-chasing) → poor cache behavior; hard to map to SIMD/PIM